

Contents

Musterlösungen — NUR für den Dozenten	1
Epic 1	1
Epic 2	2
Epic 3	4
Epic 4	5
Epic 5	6

Musterlösungen — NUR für den Dozenten

Nicht an Schüler weitergeben, bevor sie es selbst versucht haben. Lösungen sind bewusst einfach gehalten (Anfänger-Niveau), nicht “optimal” im Sinne von cleverem Kotlin — Ziel ist Nachvollziehbarkeit, nicht Eleganz.

Alle Beispiele gehen von `import framework.arena.*` aus. Hilfsfunktionen (z.B. Distanzberechnung), die in mehreren Lösungen wiederkehren, sind bei der ersten Verwendung ausgeschrieben und danach nur noch referenziert — echte Referenzimplementierungen dieser Muster liegen bereits fertig in `bots/examples/` (RandomBot, StillstandBot, ChaserBot, FluchtBot), auf die man Schüler bei Bedarf verweisen kann, ohne die Lösung direkt zu zeigen.

Epic 1

1.1 — Zufällige Bewegung

```
class MeinBot(override val name: String = "MeinBot") : RobotBrain {  
    override fun decide(sensors: Sensors): Action {  
        return Action.Move(Direction.entries.random())  
    }  
}
```

Entspricht im Kern `bots/examples/RandomBot.kt` (dort zusätzlich mit zufälliger Wahl zwischen Move/Shoot).

1.2 — Am Rand bleiben / Zentrum meiden

```
class MeinBot(override val name: String = "MeinBot") : RobotBrain {  
    override fun decide(sensors: Sensors): Action {  
        val centerX = sensors.arenaWidth / 2  
        val centerY = sensors.arenaHeight / 2  
        val pos = sensors.self.position  
  
        val nearCenter = kotlin.math.abs(pos.x - centerX) <= 1 &&  
            ↪ kotlin.math.abs(pos.y - centerY) <= 1  
        if (nearCenter) {  
            // Richtung mit dem größeren Abstand zum jeweils nächsten Rand  
            ↪ wählen  
            return if (pos.x < centerX) Action.Move(Direction.WEST) else  
                ↪ Action.Move(Direction.EAST)  
        }  
        return Action.Move(Direction.entries.random())  
    }  
}
```

1.3 — Flucht bei niedriger Gesundheit

```
class MeinBot(override val name: String = "MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val nearestEnemy = sensors.others.minByOrNull {
            ↪ distance(sensors.self.position, it.position) }
            ?: return Action.Wait

        if (sensors.self.health < 20) {
            // Weg vom Gegner: Richtung mit größerem Abstandszuwachs wählen
            val dx = sensors.self.position.x - nearestEnemy.position.x
            val dy = sensors.self.position.y - nearestEnemy.position.y
            return if (kotlin.math.abs(dx) > kotlin.math.abs(dy)) {
                Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
            } else {
                Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
            }
        }

        // Normalverhalten, z.B. aus Epic 2 (hier: einfach draufzu bewegen)
        val dx = nearestEnemy.position.x - sensors.self.position.x
        val dy = nearestEnemy.position.y - sensors.self.position.y
        return if (kotlin.math.abs(dx) > kotlin.math.abs(dy)) {
            Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
        } else {
            Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
        }
    }
}

private fun distance(a: Position, b: Position): Int =
    kotlin.math.abs(a.x - b.x) + kotlin.math.abs(a.y - b.y)
```

Vollständige, robustere Variante liegt bereits fertig in bots/examples/FluchtBot.kt.

Epic 2

2.1 — Dauerfeuer in feste Richtung

```
class MeinBot(override val name: String = "MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        return Action.Shoot(Direction.EAST)
    }
}
```

Entspricht bots/examples/StillstandBot.kt.

2.2 — Auf nächsten Gegner in Sichtlinie zielen

```
class MeinBot(override val name: String = "MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val self = sensors.self.position

        val inLineOfSight = sensors.others.filter { it.position.x == self.x ||
            ↪ it.position.y == self.y }
```

```

    val target = inLineOfSight.minByOrNull { distance(self, it.position) }
    ↪ ?: return Action.Wait

    val direction = when {
        target.position.x == self.x && target.position.y < self.y ->
    ↪ Direction.NORTH
        target.position.x == self.x && target.position.y > self.y ->
    ↪ Direction.SOUTH
        target.position.y == self.y && target.position.x > self.x ->
    ↪ Direction.EAST
        else -> Direction.WEST
    }
    return Action.Shoot(direction)
}
}

private fun distance(a: Position, b: Position): Int =
    kotlin.math.abs(a.x - b.x) + kotlin.math.abs(a.y - b.y)

```

2.3 — Gegner verfolgen, wenn nicht in Schusslinie

Kombiniert mit 2.2 — das ist im Kern die Logik von bots/examples/ChaserBot.kt:

```

class MeinBot(override val name: String = "MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val self = sensors.self.position
        if (sensors.others.isEmpty()) return Action.Wait

        val nearest = sensors.others.minByOrNull { distance(self, it.position) }
        ↪ }!!

        val inLineOfSight = nearest.position.x == self.x || nearest.position.y
        ↪ == self.y
        if (inLineOfSight) {
            val direction = when {
                nearest.position.x == self.x && nearest.position.y < self.y ->
    ↪ Direction.NORTH
                nearest.position.x == self.x && nearest.position.y > self.y ->
    ↪ Direction.SOUTH
                nearest.position.y == self.y && nearest.position.x > self.x ->
    ↪ Direction.EAST
                else -> Direction.WEST
            }
            return Action.Shoot(direction)
        }

        // Nicht in Sichtlinie: Richtung mit größerer Distanz zuerst angleichen
        val dx = nearest.position.x - self.x
        val dy = nearest.position.y - self.y
        return if (kotlin.math.abs(dx) > kotlin.math.abs(dy)) {
            Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
        } else {
            Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
        }
    }
}

```

```
}
```

```
private fun distance(a: Position, b: Position): Int =  
    kotlin.math.abs(a.x - b.x) + kotlin.math.abs(a.y - b.y)
```

Für Details und die exakte, bereits getestete Referenz: `bots/examples/ChaserBot.kt` ansehen.

Epic 3

3.1 — Einfache Zustandsmaschine

```
private enum class BotState { PATROUILLE, ANGRIFF, FLUCHT }  
  
class MeinBot(override val name: String = "MeinBot") : RobotBrain {  
    override fun decide(sensors: Sensors): Action {  
        val self = sensors.self  
        val nearest = sensors.others.minByOrNull { distance(self.position,  
            ↪ it.position) }  
  
        val state = when {  
            self.health < 20 && nearest != null -> BotState.FLUCHT  
            nearest != null -> BotState.ANGRIFF  
            else -> BotState.PATROUILLE  
        }  
  
        return when (state) {  
            BotState.FLUCHT -> {  
                val dx = self.position.x - nearest!!.position.x  
                val dy = self.position.y - nearest.position.y  
                if (kotlin.math.abs(dx) > kotlin.math.abs(dy))  
                    Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)  
                else  
                    Action.Move(if (dy > 0) Direction.SOUTH else  
            ↪ Direction.NORTH)  
            }  
            BotState.ANGRIFF -> {  
                val inLineOfSight = nearest!!.position.x == self.position.x ||  
                ↪ nearest.position.y == self.position.y  
                if (inLineOfSight) {  
                    val direction = when {  
                        nearest.position.x == self.position.x &&  
            ↪ nearest.position.y < self.position.y -> Direction.NORTH  
                        nearest.position.x == self.position.x -> Direction.SOUTH  
                        nearest.position.x > self.position.x -> Direction.EAST  
                        else -> Direction.WEST  
                    }  
                    Action.Shoot(direction)  
                } else {  
                    val dx = nearest.position.x - self.position.x  
                    val dy = nearest.position.y - self.position.y  
                    if (kotlin.math.abs(dx) > kotlin.math.abs(dy))  
                        Action.Move(if (dx > 0) Direction.EAST else  
            ↪ Direction.WEST)  
                    else
```

```

        Action.Move(if (dy > 0) Direction.SOUTH else
↪ Direction.NORTH)
    }
}
BotState.PATROUILLE -> Action.Move(Direction.entries.random())
}
}
}

```

```

private fun distance(a: Position, b: Position): Int =
    kotlin.math.abs(a.x - b.x) + kotlin.math.abs(a.y - b.y)

```

3.2 — Zielpriorisierung (schwächster Gegner zuerst)

```

class MeinBot(override val name: String = "MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        // Kriterium: niedrigste HP zuerst (leichtestes Ziel zum Ausschalten)
        val target = sensors.others.minByOrNull { it.health } ?: return
        ↪ Action.Wait
        // ... danach wie 2.2/2.3 in Richtung von `target` schießen/bewegen
        return Action.Wait // Platzhalter – Kombination mit 2.2/2.3 ergänzen
    }
}

```

Hinweis für Schüler: der eigentliche Lernpunkt hier ist die Auswahl-Regel (`minByOrNull { it.health }` statt `minByOrNull { distance }`), nicht die Bewegungs-/Schuss-Logik selbst — die kann 1:1 aus Epic 2 übernommen werden.

3.3 — Kür-Aufgabe

Keine feste Musterlösung (bewusst offen). Bei Rückfragen: prüfen, ob die Idee mit `Sensors/Action` ausdrückbar ist (kein Bot kann z.B. "sehen", was ein Gegner als Nächstes tun wird — nur den aktuellen `RobotState` aller anderen).

Epic 4

4.1 — Unit-Test für eigene Entscheidungslogik

```

package bots.teama

```

```

import framework.arena.*
import kotlin.test.Test
import kotlin.test.assertEquals

```

```

class MeinBotTest {
    @Test
    fun `schießt wenn Gegner in gleicher Reihe steht`() {
        val self = RobotState(id = "bot-0", teamName = "Team A", position =
        ↪ Position(2, 5), health = 100)
        val enemy = RobotState(id = "bot-1", teamName = "Team B", position =
        ↪ Position(7, 5), health = 100)
        val sensors = Sensors(self = self, others = listOf(enemy), arenaWidth
        ↪ = 10, arenaHeight = 10, tick = 1)
    }
}

```

```

        val action = MeinBot().decide(sensors)

        assertEquals(Action.Shoot(Direction.EAST), action)
    }
}

```

Wichtig: Test-Datei muss unter `src/test/kotlin/bots/teamX/...` liegen (Ordner ggf. neu anlegen), damit Gradle sie findet.

4.2 — Testduell protokollieren

Keine Code-Lösung nötig — organisatorische Story. Hinweis für den Dozenten: einfach in der App zwei Bots auswählen (eigener Bot + ein Bot aus `bots/examples`), Start drücken, Ergebnis im Scoreboard/Log ablesen lassen.

Epic 5

Beide Storys sind rein organisatorisch, keine Musterlösung nötig. Bei 5.1 sicherstellen, dass Teams nicht aus Versehen mehrere widersprüchliche `MeinBot`-Varianten gleichzeitig in `teamXBots` eingetragen lassen, falls sie mehrere Ausbaustufen parallel behalten haben.